PyGameMaker — Tutorial 15: Fruit Fusion — Teacher Guide

Companion for the in-app tutorial (**Help > Tutorials > Fruit Fusion: Catch and Merge!**, 6 pages) and the matching student handout and worksheet. No earlier tutorial is required — this is a good first project, including for the youngest students (8-9), since it never requires reading dense instructions unassisted: every phase repeats the same four-step recipe with new names and numbers.

Overview

Students build a "merge" game in the style of popular mobile games (catch a matching fruit, it fuses into something bigger) without any real stacking physics. Four phases: a moving basket; a first fusion (cherry to strawberry); a second fusion (strawberry to orange), built by literally repeating the first fusion's recipe; and a final fusion (orange to watermelon) that also wins the game. New ideas: a **variable that remembers state** (held_level), an **If / Else** block, and **changing an object's sprite at runtime**.

> Ask "how does the basket remember what it's holding?" The answer, "a number we invent ourselves, held_level", is the key idea of the lesson — and the reason there is no "empty basket" state to explain (it starts holding a cherry, i.e. held_level = 1).

Suggested Timing (45 minutes)

The tutorial says 20-25 minutes for a confident student; allow extra time for the first If/Else block, since it is new.

Segment	Time	What happens
Recap and predict	5 min	Show the finished game; ask how the basket could "remember" what it's holding
Phase 1: moving basket	5-10 min	Page 2
Phase 2: first fusion	10-15 min	Page 3 (the slowest page — the If/Else is new)
Phase 3: second fusion	5-10 min	Page 4 (same recipe, should be faster)
Phase 4: winning	10 min	Page 5
Worksheet / exit ticket	5 min	Worksheet Parts A-C

Step-by-Step Walkthrough and Common Issues

Phase 1: moving basket

- Three keyboard events, exactly as in Tutorial 2 / Tutorial 9. The basket may leave the screen; that is expected and not fixed in this tutorial.
- The sprite is spr_basket_empty at this point — no fusion concept has been introduced yet, so there is nothing to hold.

Phase 2: first fusion

- This is the phase that introduces every new idea at once: held_level, the basket's starting sprite swap to spr_basket_cherry, and the collision's If/Else. Budget the most time here.
- **The basket's sprite changes to spr_basket_cherry BEFORE any gameplay** — this is a one-time manual sprite swap in the object's properties, not an action. A student who skips it will have a basket that never visually shows what it's holding, even though the logic underneath is correct; the game will still score correctly, which can make the mistake easy to miss.
- held_level is set in **Create**, not anywhere else. A student who sets it in the spawner or the fruit object by mistake will see it reset unexpectedly.
- The collision's *If held_level equals 1* branch must contain all three actions (set held_level, add score, set sprite) — a student who puts the score or sprite action in the wrong branch (or outside the If entirely) gets a game that scores correctly but never shows the fusion, or shows it on every catch including mismatches.
- *Destroy other instance*, not *Destroy this instance* — the common one-letter mistake that destroys the basket instead of the caught fruit.

Phase 3: second fusion

- Deliberately the fast phase: the exact same four steps as Phase 2, with "cherry" renamed to "strawberry" and 1/2 changed to 2/3. If a student struggled with Phase 2's If/Else, this is where it clicks, because they're building the identical shape again with different names.
- A frequent copy-paste slip: forgetting to change the *If held_level equals 1* condition to *equals 2*. The symptom is a strawberry catch doing nothing useful (held_level is never 1 once a cherry has already fused), which looks like a bigger bug than it is.

Phase 4: winning

- The win condition is **not** a separate check anywhere — Go to room room_win is one more action inside the orange collision's *If* branch, run exactly once, at the moment the fourth fusion happens. Students coming from Tutorial 9's separate Step-event win check sometimes go looking for an equivalent here and don't need to.
- There is deliberately no falling watermelon sprite or spawner. If a student asks "where's the watermelon fruit", the answer is: it only ever exists as the basket's final look, never as something that falls.
- obj_win_text draws its message at fixed coordinates in the 640x480 room this tutorial uses throughout (not the 1024x768 default some other tutorials use) — a student who typed the Tutorial 9 coordinates from memory will see text that's off-screen or badly placed.

> **Reference projects.** Download the checkpoint projects from the wiki (15_fruit_fusion_checkpoints.zip): one project for the end of each phase. Give a stuck student the previous phase rather than troubleshooting their If/Else from scratch.

Vocabulary Introduced

Term	What it means here
Held fruit	The one fruit the basket is currently carrying, remembered in held_level
Fusion	Catching a matching fruit so it grows into the next, bigger fruit
If / Else	A block that does one thing when something is true, and a different thing when it is not
Spawner	An invisible object whose only job is to create other objects on a timer

Sprite swap at runtime	Changing which picture an object shows while the game is running, as feedback

Discussion Questions

- Why does the basket need a variable (held_level) instead of just checking its current sprite?
- What would change if the fruit types fell from a single spawner that picked a random type, instead of three separate spawners?
- Why can mismatching never end the game in this version? What would change if it could?
- What other games use "combine two matching things to make a bigger thing" as their core idea?

Differentiation

- **Support:** give the Phase 2 checkpoint, so a struggling student only has to build the second and third fusion by pattern-matching the first.
- **Extension:** a fourth spawner and fruit tier between orange and watermelon; lives that go down on repeated mismatches; a second basket for two-player competitive fusing.

Worksheet Answer Key

Part A: 1-C, 2-D, 3-A, 4-B.

Part B: 1. obj_player, Collision with obj_fruit_cherry. 2. obj_spawn_cherry, Alarm 0. 3. obj_fruit_cherry (and obj_fruit_strawberry, obj_fruit_orange), Outside Room. 4. obj_win_text, When drawing. 5. obj_win_text, Key press: space.

Part C:

1. The collision's *If* branch only checks held_level == 3 (holding an orange); if held_level is something else, the *If* is false and the *Otherwise* branch runs instead — the fruit's own type never changes, only what you currently hold does, and a cherry can only ever match a basket already holding a cherry.
2. The watermelon fusion would never trigger: a correct match would still add 50 points and swap the sprite only if those actions were (also) left in the *If* branch, but *Go to room room_win* sitting in *Otherwise* would instead fire on every MISMATCH, sending the player to the win room without ever actually completing the fusion — the opposite of the intended behaviour.
3. One spawner per type means the collision event for that exact object (obj_fruit_cherry, say) already identifies the tier, with no extra conditional needed. A single random spawner would need the spawned fruit to carry its own "which tier am I" variable, and every collision check would need to read other's variable before comparing it to held_level — more moving parts for the same result.

Parts D and E: completion and reflection.

Rubric: Fruit Fusion

Level	What the game shows
4 - Complete	The basket moves; three fruit types fall and are each handled by their own collision; a correct catch fuses the basket up one tier with the right score bonus and sprite change; a mismatch only adds 1 point; fusing an orange shows YOU WIN! and SPACE restarts
3 - Working	Moving and the first fusion work; the second or third fusion, or the win screen, is missing or incorrect

2 - Partly there	The basket moves and fruit falls, but no fusion works correctly
1 - Started	The basket exists and moves; nothing falls

End-of-Session Checklist

- [ ] Every student has a saved project (Ctrl+S)
- [ ] Every student reached at least Phase 2 (one fusion works end to end)
- [ ] Note who needs the checkpoint project next time